Ripasso di Arduino e funzioni con parametri

Lezione 1 — 2MAK, SST-INF-01c — Firmware: cosa ricordare dell'anno scorso, e le funzioni

15/10/2026

Nota

Questa dispensa è anche il **riepilogo** di quanto fatto l'anno scorso sul firmware: chi è arrivato quest'anno trova qui tutto quello che serve per allinearsi. Il **29 ottobre** (mattina) è prevista una **verifica di ripasso** del firmware, di circa due ore, su tutti gli argomenti dell'anno scorso e su quelli di questa lezione.

1 La struttura di un programma Arduino

Un programma Arduino ha due blocchi fondamentali:

- `setup()`: contiene le **configurazioni iniziali**, istruzioni eseguite **una sola volta**, all'accensione;
- `loop()`: contiene la parte che deve essere eseguita **continuamente**, ciclicamente, finché la scheda resta accesa.

2 Ingressi e uscite: `pinMode`, `digitalWrite`, `digitalRead`

Esempio

Le porte del supermercato. In molti supermercati le porte sono a senso unico: da una si entra, da un'altra si esce. Anche Arduino ha delle "porte", i **piedini**: da alcuni le informazioni **entrano** (**input**), da altri **escono** (**output**). Se è un'uscita, è Arduino a decidere cosa far uscire: **acceso o spento**. Se è un'entrata, Arduino non decide cosa arriva: è il **mondo esterno** a dire se sul piedino c'è un segnale oppure no.
Un altro modo di vederlo: per parlare usi la **bocca**, per ascoltare le **orecchie**. Non puoi parlare con le orecchie né ascoltare con la bocca. Se il piedino 12 è un input, puoi "sentire" cosa dice il mondo esterno sul 12, ma non puoi decidere tu cosa deve arrivarci.

Definizione

`pinMode(piedino, OUTPUT)` oppure `pinMode(piedino, INPUT)` decide la **direzione** di un piedino: se è un'uscita o un'entrata. Decide *solo* la direzione, non cosa passa.

Nota

Di solito `pinMode` si scrive **all'inizio**, dentro `setup()`: una volta deciso che un piedino è un'uscita, resta un'uscita. Nei prodotti più evoluti ci sono piedini che per un certo tempo fanno da

input e per un altro da output, e allora `pinMode` si mette anche dentro `loop()`, nei punti in cui serve cambiare. Lo vedremo eventualmente il prossimo anno.

Definizione

Su un piedino configurato come **output**, l'unica cosa che si può fare è decidere se far uscire tensione oppure no: **`digitalWrite(piedino, HIGH)`** (acceso) oppure **`digitalWrite(piedino, LOW)`** (spento).
Su un piedino configurato come **input** si "ascolta" con **`digitalRead(piedino)`**, che dice se sul piedino c'è un livello alto o basso. Il valore letto va **messo da qualche parte**, di solito in una variabile:

```
int x = digitalRead(12);
```

Attenzione

Questi tre comandi — `pinMode` (direzione), `digitalWrite` (scrivere su un output), `digitalRead` (leggere un input) — devono essere **solidi** nella vostra memoria: sono la base di tutto quello che faremo.

3 Il tempo: `delay` e `delayMicroseconds`

Definizione

`delay(t)` **blocca** la macchina per un certo tempo: resta ferma per `t` **millisecondi**. `delay(1000)` significa fermarsi per 1000 millesimi di secondo, cioè **un secondo**. Il valore minimo utile è 1 (un millisecondo): `delay` accetta solo **numeri interi**, quindi non si può scrivere `delay(0.5)` per mezzo millisecondo.

Nota

Per tempi più brevi di un millisecondo esiste **`delayMicroseconds(t)`**, che funziona allo stesso modo ma esprime il tempo in **microsecondi** (milionesimi di secondo): `delayMicroseconds(1000)` ferma la macchina per un millesimo di secondo.
Precisazione rispetto alla lezione: se si scrive comunque `delay(0.5)`, il programma viene compilato, ma il valore viene **troncato** a 0 e la pausa non c'è.

4 La comunicazione seriale: `Serial`

`Serial` è un insieme (una "classe") di comandi che gestisce la **comunicazione** tra Arduino e computer. Quelli visti l'anno scorso:

- **`Serial.begin(velocità)`**: fissa la **velocità** con cui si scambiano i dati. Arduino e il computer (o qualunque dispositivo che riceve) devono essere impostati alla **stessa velocità**, altrimenti non si capiscono: non avviene in automatico, lo dovete impostare voi. I valori tipici sono **9600** e **115200**. Non serve andare sempre al massimo.

- **`Serial.print(...)`** invia un messaggio alla console di Arduino; il messaggio successivo continua **sulla stessa riga**.
- **`Serial.println(...)`** fa lo stesso, ma **va a capo** dopo il messaggio (*ln* = *line*).
- **`Serial.available()`** dice **quanti messaggi** (caratteri) sono arrivati e sono **in attesa** di essere letti.
- **`Serial.read()`** **legge** il primo carattere in attesa e lo **toglie dalla coda**.

Esempio

Lo sportello e la coda. Quando il computer manda dati ad Arduino, i dati finiscono in una memoria chiamata **buffer**. È come un ufficio con uno sportello: chi arriva mentre lo sportello è occupato si mette **in coda** e aspetta il suo turno. L'invio non dipende da Arduino, ma da chi manda i dati: Arduino può essere veloce quanto si vuole, ma gli serve tempo per elaborare ogni dato, quindi deve poter tenere da parte quelli che arrivano nel frattempo. `Serial.available()` dice **quanta gente c'è in coda**; `Serial.read()` fa **passare allo sportello** il primo della coda.

Esempio

Un programma tipico: Arduino scrive continuamente "ciao" sulla console; se dal computer arriva la lettera `x`, risponde "premuto".

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println("ciao");
  if (Serial.available() > 0) {   // c'e' qualcuno in coda?
    char dato = Serial.read();    // lo leggo e lo tolgo dalla coda
    if (dato == 'x') {
      Serial.println("premuto");
    }
  }
  delay(500);   // senza una pausa i messaggi correrebbero troppo veloci
}
```

In base al valore di `dato`, il programma decide cosa fare: dipende da cosa vogliamo realizzare.

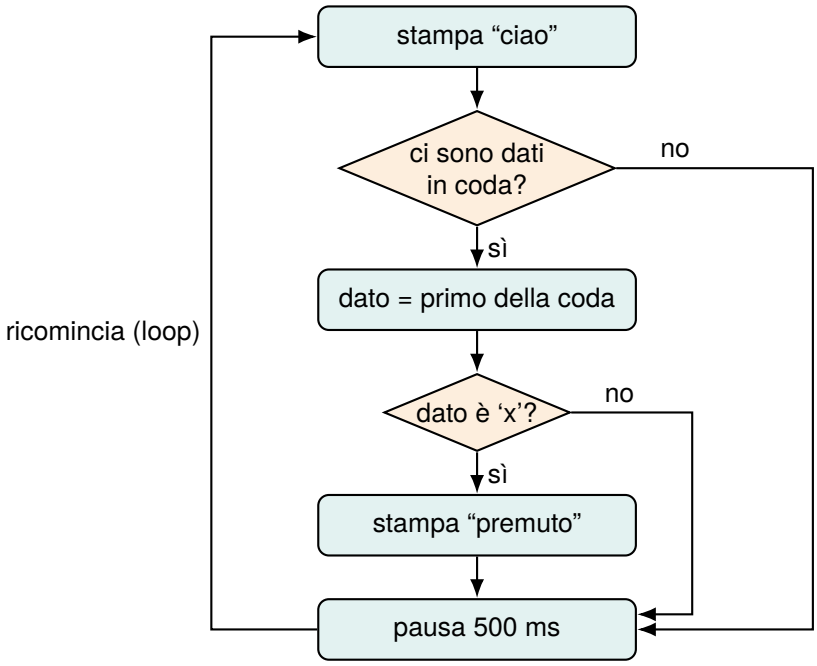


Figura 1: Il diagramma di flusso del programma “ciao / premuto”: a ogni giro di `loop()` stampa “ciao” e, solo se nella coda c’è una `x`, risponde “premuto”.

5 Ripasso di C: tipi di dato e costrutti

Definizione

Bit e byte: un **byte** è fatto da **8 bit**.
Tipi di dato su Arduino Uno:

Tipo	Byte	Valori
`char`	1	da -128 a 127 (un carattere, o un numero piccolo)
`byte` (`unsigned char`)	1	da 0 a 255
`int`	2	da $-32\,768$ a $32\,767$
`float`	4	numeri **con la virgola**, circa 6–7 cifre significative

Attenzione

Quanti byte, non quanti valori. Alla domanda “di quanti byte è fatto un `char`?” la risposta è **1 byte**, cioè 8 bit — non “255”, che riguarda invece *quali valori* ci stanno dentro. E attenzione all’intervallo: su Arduino (e sul PC) il `char` è **con segno**, quindi va da -128 a 127; l’intervallo da 0 a 255 è quello di `byte` (`unsigned char`). La dimensione di `int` **dipende dalla macchina**: su Arduino Uno sono 2 byte, sul computer che avete davanti di solito 4.

Definizione

`if` permette di **prendere una decisione**: se succede questo, allora fai quello (“se piove, prendo l’ombrello”).
I **costrutti di ripetizione** sono due: **`for`** e **`while`**. Con l’uno si può fare tutto quello che si fa con l’altro: la scelta è una questione di **stile**. Di solito i programmatori usano:

- `for` quando sanno **in anticipo** quante volte ripetere (“accendi il LED 10 volte”);
- `while` quando il numero di ripetizioni **non si conosce prima** e dipende da eventi esterni, cioè si scopre solo *durante* l’esecuzione (a *runtime*).

Attenzione

Questi argomenti vanno **ripassati adesso**: nelle prossime lezioni il `for`, le variabili e i tipi di dato si usano direttamente, senza rispiegarli ogni volta. Saranno anche nella **verifica di ripasso** del 29 ottobre.

6 Le funzioni

Esempio

La mossa speciale dei videogiochi. Nei vecchi videogiochi di combattimento, per fare una mossa speciale bisognava premere velocemente una sequenza di tasti — per esempio basso, basso, sinistra, destra, pugno, calcio — e serviva una certa abilità manuale. Oggi molti controller permettono di assegnare **l’intera sequenza** a **un solo pulsante**: premi quel pulsante e la mossa parte tutta.
Una **funzione** è esattamente questo: una **raccolta di operazioni** a cui si dà un **nome**. Ogni volta che nel programma si scrive quel nome, la macchina esegue tutte le operazioni della raccolta, senza doverle riscrivere.

Esempio

Cinque euro e un panino. Immaginate di avere un aiutante a cui si chiede una sola cosa: gli date 5 euro e, dopo un po’, torna con un panino. Come lo fa (va alla macchinetta, oppure va in cucina, taglia il pane, mette l’affettato e il formaggio, aggiunge la salsa) a voi **non interessa**: qualcuno gliel’ha spiegato una volta per tutte. A voi interessa solo che **date 5 euro** e **ricevete un panino**.
Una funzione in C si scrive allo stesso modo:

```
Panino comprapanino(int soldi) {   // schema per capire, non codice da compilare
  // va in cucina, taglia il pane, ... prepara il panino
}
```

- a sinistra c’è **che cosa riconsegna** la funzione (il **tipo** del risultato: qui un panino);
- poi il **nome** che diamo alla funzione;
- tra parentesi tonde **che cosa le consegniamo** noi (i **parametri**: qui i soldi);
- tra le graffe **come lo fa**.

Definizione

Anche `setup` e `loop` sono funzioni: `void setup()`. `void` significa “vuoto”: la funzione **non**

riconsegna niente, deve solo eseguire delle cose. E le parentesi vuote dicono che non le consegniamo **nessun parametro**. Lo stesso vale per `loop`.

6.1 Una funzione che fa lampeggiare un LED

Un lampeggio si fa con `digitalWrite` alto, `delay`, `digitalWrite` basso, `delay`. Lo possiamo racchiudere in una funzione: non deve riconsegnare niente, quindi è `void`.

```
void lampeggio() {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

Nel `loop` basta scrivere **`lampeggio();`**: quando il programma arriva a quel punto, **salta** dentro la funzione, esegue le quattro istruzioni e poi **riprende** da dove era rimasto. È come incastrare nel codice un pezzettino che si può richiamare quando serve.

6.2 Aggiungere parametri: quale piedino, quante volte, quanto veloce

Così la funzione fa lampeggiare *sempre* il piedino 13. Possiamo **personalizzarla** dandole delle informazioni, cioè dei **parametri**:

```
void lampeggio(int pin) {
  digitalWrite(pin, HIGH);
  delay(500);
  digitalWrite(pin, LOW);
  delay(500);
}
```

Scrivendo **`lampeggio(13);`** il numero 13 viene **copiato** dentro `pin`, e lampeggia il piedino 13; scrivendo subito sotto **`lampeggio(7);`** lampeggia il piedino 7. Il codice della funzione non è cambiato: è cambiato il **parametro**, scelto da fuori. Il nome `pin` l'abbiamo scelto noi: poteva essere qualunque altro nome.

Un passo in più: anche **quante volte** lampeggiare. Qui serve il `for` (ecco perché va ripassato):

```
void lampeggio(int pin, int n) {
  for (int i = 0; i < n; i++) {
    digitalWrite(pin, HIGH);
    delay(500);
    digitalWrite(pin, LOW);
    delay(500);
  }
}
```

Nota

Nel `for`: `int i = 0` è la **condizione iniziale**; `i < n` è la **condizione di ripetizione** (si ripete finché `i` è minore di `n`); `i++` **incrementa** `i`. Quindi `i` vale 0, 1, 2, ... fino a $n-1$: il blocco viene

eseguito esattamente `n` volte.

Con due parametri, separati da una **virgola**, nel `loop` si scrive `lampeggio(13, 3);` e `lampeggio(7, 9);`: il primo numero finisce in `pin`, il secondo in `n`. Il piedino 13 lampeggia 3 volte, il piedino 7 lampeggia 9 volte.

Esempio

Quanto codice si risparmia. Senza funzioni, per far lampeggiare 3 volte il piedino 13 e 9 volte il piedino 7 bisognava scrivere le quattro istruzioni del lampeggio dentro due `for` diversi, circa 10 righe; con la funzione bastano le sue righe più due chiamate. E all'aumentare delle richieste **il codice non cresce**: per far lampeggiare il piedino 5 per 100 volte basta aggiungere `lampeggio(5, 100);`.

Nota

La funzione si scrive **fuori** da `setup()` e da `loop()`: prima o dopo, ma mai dentro. Nell'ambiente Arduino (e in Tinkercad) può stare anche dopo `loop()`, come nel codice della lezione.

6.3 Funzioni che riconsegnano un risultato

`lampeggio` non riconsegna niente. Una funzione può invece fare un **calcolo** e **riconsegnare il risultato**: allora a sinistra del nome, al posto di `void`, si scrive il **tipo del dato** riconsegnato (`char`, `int`, `float`, ...).

Esempio

La distanza tra due punti. Nel piano cartesiano abbiamo due punti, (x_1, y_1) e (x_2, y_2). Tracciando le proiezioni sugli assi si forma un triangolo rettangolo: i due **cateti** misurano $x_2 - x_1$ e $y_2 - y_1$, e la distanza tra i punti è l'**ipotenusa**. Con il **teorema di Pitagora**:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Una funzione che fa questo calcolo:

```
float calcolo(int x1, int y1, int x2, int y2) {
  int dx = x2 - x1;              // delta x: la differenza
  int dy = y2 - y1;              // delta y
  int ipotenusa2 = dx*dx + dy*dy; // ipotenusa al quadrato
  float r = sqrt(ipotenusa2);    // radice quadrata
  return r;                      // riconsegna il risultato
}
```

Il comando **`return`** va messo alla fine della funzione e dice **quale valore riconsegnare**. In C la radice quadrata si scrive **`sqrt`**.
Nel `loop`: `float distanza = calcolo(2, 1, 6, 4);`. Il primo punto è $(2, 1)$, il secondo $(6, 4)$. La macchina calcola $dx = 6 - 2 = 4$, $dy = 4 - 1 = 3$, poi $4 \cdot 4 + 3 \cdot 3 = 16 + 9 = 25$, e infine $\sqrt{25} = 5$: nella variabile `distanza` finisce **5**.

Nota

Il bello delle funzioni: una procedura **lunga e complessa** viene scritta **una volta sola**, e chi la usa non deve ricordarsi come si fa. Funzioni come `sqrt`, il seno e il coseno sono state scritte da qualcun altro, magari decenni fa, e da allora le usiamo senza riscriverle: chi programma una calcolatrice non riscrive il calcolo del seno, usa la funzione che esiste già.

Nota

Esistono anche funzioni che **riconsegnano un valore** senza bisogno di **nessun parametro**: per esempio `millis()`, che dice quanti millisecondi sono passati dall'accensione. In lezione si è fatto l'esempio della funzione che genera un **numero casuale**: *precisazione*: su Arduino si chiama `random` e vuole almeno un parametro, il limite (`random(10)` dà un numero da 0 a 9).

7 Le etichette: `#define`

Nel circuito della lezione il LED rosso è sul piedino 3 e il verde sul 5. Scrivendo `pinMode(3, OUTPUT)`, `digitalWrite(5, HIGH)`, ... bisogna **ricordarsi** ogni volta chi è il 3 e chi è il 5. Esiste la possibilità di dare un **nome** a un numero:

```
#define RED_LED 3
#define GREEN_LED 5
```

Da quel momento si può scrivere `pinMode(RED_LED, OUTPUT)` al posto di `pinMode(3, OUTPUT)`.

Definizione

`#define` crea un'**etichetta** (*label*): **non è una variabile**, è un nome dato a un numero, che prima della compilazione viene sostituito dal numero stesso. Non va né il segno = né il punto e virgola finale.

Esempio

Perché conviene. Con poche righe di codice il vantaggio non si vede. Ma in un programma da 10 000 righe, se alla riga 7422 trovate `digitalWrite(14, HIGH)`, il 14 che LED è? Verde, blu, bianco? Bisogna tornare in cima o guardare il circuito. Se invece trovate `digitalWrite(LED_BIANCO, HIGH)`, è chiaro subito.
È come la **rubrica del telefono**: per chiamare qualcuno non cercate il suo numero, cercate il suo **nome**. Lo stesso per numeri lunghi e difficili da ricordare, come pi greco: gli si dà un nome e lo si usa al posto del numero. Per pi greco, 3,14 è poco preciso: per calcolare la traiettoria di un proiettile verso la Luna servono parecchie cifre, verso Marte ancora di più, perché più lontano si va più il calcolo deve essere preciso. Con un'etichetta il numero lungo si scrive una volta sola.

Attenzione

- Nel nome si usa il **trattino basso** (`RED_LED`), **non il meno**: `RED-LED` verrebbe letto come la sottrazione tra `RED` e `LED`.
- Per convenzione le etichette si scrivono **in maiuscolo**. Come nelle email e nei messaggi, dove il maiuscolo vuol dire "urlare", anche in C il maiuscolo ha un significato preciso. In C

il maiuscolo indica un valore **costante**, che non cambia mai; il minuscolo indica qualcosa che può cambiare durante l'esecuzione (come le variabili). Funziona anche in minuscolo, ma il buon stile richiede il maiuscolo.

8 Esercitazione su Tinkercad: due LED, tre parametri

Sicurezza sul lavoro

Il circuito si costruisce in **simulazione** (Tinkercad), ma le regole sono quelle del circuito reale: un filo che collega un **piedino di output** direttamente **ai 5 V** crea un **cortocircuito** quando il piedino va a livello basso, e su una scheda vera può **danneggiare Arduino**. Seguite sempre lo schema.

Il circuito. In Tinkercad si porta dentro un **Arduino Uno** e una **breadboard**, si collegano come sempre le linee di alimentazione (5 V e GND) e si mettono due LED: **rosso sul piedino 3**, **verde sul piedino 5**. Ogni LED ha il suo **resistore da 220 Ω**: il piedino va al resistore, il resistore all'**anodo** del LED, il **catodo** va a GND.

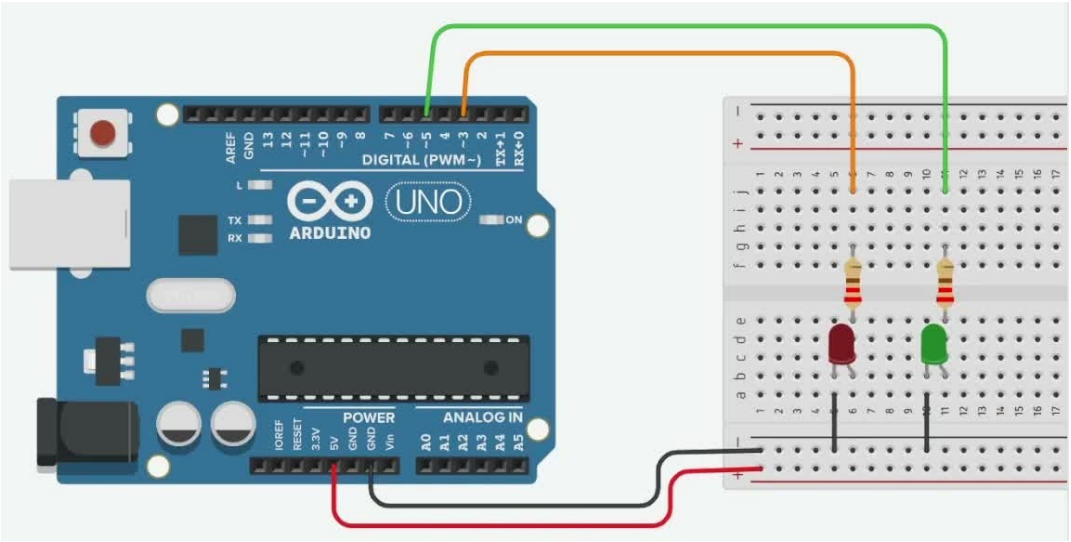


Figura 2: Il circuito su Tinkercad: il filo arancione porta il piedino 3 al LED rosso, il filo verde porta il piedino 5 al LED verde; ogni LED ha il suo resistore, e i catodi sono collegati alla linea GND della breadboard.

Attenzione

Errori visti durante l'esercitazione:

- un **LED montato al contrario** (anodo e catodo scambiati) **non si accende**: l'anodo va verso il resistore e il piedino, il catodo verso GND;
- **fili in più** tra il resistore e i 5 V, che non c'entrano con lo schema: oltre a non servire, creano il cortocircuito descritto sopra. Confrontate sempre il vostro disegno con lo schema.

Il codice. Prima con due parametri (`lampeggio(RED_LED, 7)` e `lampeggio(GREEN_LED, 3)`): il LED

rosso fa 7 lampeggi, poi il verde ne fa 3, poi si ricomincia. Poi la richiesta: aggiungere un **terzo parametro**, il **tempo** `T`, così che il rosso lampeggi 7 volte veloce (200 ms) e il verde 3 volte lento (700 ms). La soluzione:

```
/*
    sul pin 3 abbiamo il led rosso
    sul pin 5 abbiamo il led verde
*/
#define RED_LED 3
#define GREEN_LED 5

void setup()
{
    pinMode(RED_LED, OUTPUT);
    pinMode(GREEN_LED, OUTPUT);
}

void loop()
{
    lampeggio(RED_LED, 7, 200);
    lampeggio(GREEN_LED, 3, 700);
}

void lampeggio(int pin, int N, int T)
{
    for (int i = 0; i < N; i++)
    {
        digitalWrite(pin, HIGH);
        delay(T);
        digitalWrite(pin, LOW);
        delay(T);
    }
}
```

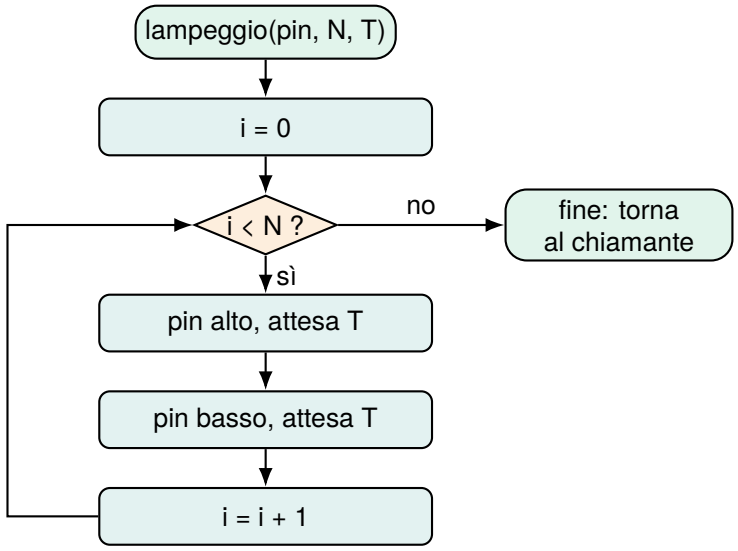


Figura 3: Il diagramma di flusso della funzione `lampeggio(pin, N, T)`: il blocco acceso/spento viene ripetuto N volte, ciascuna fase dura T millisecondi.

Eseguendo la simulazione, il LED rosso lampeggia velocemente 7 volte e il verde lentamente 3 volte. Rispetto alla versione precedente si sono aggiunti solo **pochi pezzettini**: la struttura del codice è rimasta la stessa.

Nota

Perché i due LED non lampeggiano insieme? Perché il programma esegue le istruzioni **una dopo l'altra**: finché la funzione sta facendo lampeggiare il rosso, non può occuparsi del verde. Far lavorare le due cose "contemporaneamente" richiede un modo di scrivere il codice completamente diverso: lo affronteremo il prossimo anno.

Nota

In simulazione si nota che il LED, quando si spegne, impiega un attimo: anche nella realtà un LED ha un piccolo **tempo di spegnimento**.

9 Cosa ricordare di questa lezione

- Un programma Arduino ha **`setup()`** (eseguito una volta) e **`loop()`** (eseguito di continuo).
- **`pinMode`** decide la direzione di un piedino; **`digitalWrite`** scrive su un output; **`digitalRead`** legge un input. **`delay`** ferma la macchina per un tempo in millisecondi (solo interi).
- **`Serial`**: `begin` (stessa velocità ai due capi), `print`/`println`, `available` (quanti in coda nel buffer), `read` (legge e toglie dalla coda).
- Tipi di dato: `char` 1 byte ($-128 \ldots 127$), `int` 2 byte su Arduino Uno, `float` 4 byte con la virgola. `for` se il numero di ripetizioni è noto prima, `while` se dipende da eventi esterni.
- Una **funzione** è una raccolta di operazioni con un nome: tipo riconsegnato (o `void`), nome, **parametri** tra parentesi, e **`return`** per riconsegnare il risultato.
- Con i parametri la stessa funzione serve in tanti casi: `lampeggio(pin, N, T)` fa lampeggiare qualunque piedino, quante volte e alla velocità che si vuole.
- **`#define`** dà un nome a un numero: si scrive in maiuscolo, con il trattino basso, senza `=` e senza punto e virgola.

Esempio

Per la verifica di ripasso del 29 ottobre ripassate tutto quanto sopra: comandi di base di Arduino, `Serial`, variabili e tipi di dato, `if`, `for` e `while`, funzioni con parametri e con valore riconsegnato. Nella verifica di elettronica successiva ci sarà anche una domanda sul **perché** il resistore del LED è da **220 Ω**: ripassatelo.